



TFAS: zero-shot NAS for general time-series analysis with time-frequency aware scoring

Patara Trirat¹ · Jae-Gil Lee²

Received: 11 April 2025 / Revised: 25 June 2025 / Accepted: 29 June 2025

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2025

Abstract

Designing effective neural networks from scratch for various time-series analysis tasks, such as activity recognition, fault detection, and traffic forecasting, is time-consuming and heavily relies on human labor. To reduce the reliance on human labor, recent studies adopt neural architecture search (NAS) to design neural networks for time series automatically. Still, existing NAS frameworks for time series only focus on one specific analysis, such as forecasting and classification, with expensive search methods. This paper therefore aims to build a *unified* zero-shot NAS framework that effectively searches neural architectures for a variety of tasks and time-series data. However, to build a general framework for different tasks, we need a zero-shot proxy that consistently correlates with the downstream performance across different characteristics of time-series datasets. To address these challenges, we propose a zero-shot NAS framework with novel **Time-Frequency Aware Scoring** for general time-series analysis, named **TFAS**. To incorporate **TFAS** into existing foundation time-series models, we adopt time-frequency decomposition methods and introduce the concept of *augmented architecture* to the foundation models. This augmented architecture enables the zero-shot proxy to be aware of the decomposed time and frequency information, resulting a more accurate estimation of downstream performance for particular datasets. Empirically, we show that the architectures found by **TFAS** gain improvement of up to 23.6% over state-of-the-art hand-crafted baselines in five mainstream time-series data mining tasks, including short- and long-term forecasting, classification, anomaly detection, and imputation.

Keywords AutoML · Neural architecture search · Zero-shot proxy · Time-series analysis · Time-frequency decomposition

Editors: Ana Carolina Lorena, Maria Concepcion Bielza Lozoya, Tiago Agostinho de Almeida.

✉ Patara Trirat
patara@deepauto.ai

✉ Jae-Gil Lee
jaegil@kaist.ac.kr

¹ DeepAuto.ai, Seoul, Republic of Korea

² School of Computing, KAIST, Daejeon, Republic of Korea

1 Introduction

Time-series analysis is increasingly essential and at the core of various real-world applications, such as forecasting for accident risk prediction (Trirat et al., 2023), classification for human activity recognition (Shin et al., 2023), anomaly detection for industrial system monitoring (Trirat et al., 2023), and imputation of missing values for data preprocessing (Miao et al., 2021). Due to the proliferation of sensor devices and the practical value of time-series analysis, many papers (Lim and Zohren, 2021; Choi et al., 2021; Wen et al., 2023) have proposed deep neural networks for several time-series analysis tasks by exploiting the powerful non-linear modeling capability. The goal is to learn complex temporal variations in real-world time series that have multiple intricate patterns (e.g., declining, increasing, and oscillating) mixing and overlapping with each other.

However, making efficient deep models for various downstream analysis tasks is extremely challenging because it laboriously relies on tedious manual trial-and-error to design the network architectures and select corresponding hyperparameters. Also, the inefficient hand-crafted design will inevitably involve human bias leading to sub-optimal architectures (Zhang et al., 2023). To address this problem, recent endeavors employ neural architecture search (NAS) to automatically discover an optimal architecture for a specific time-series analysis task, such as forecasting (Deng et al., 2022) and classification (Ren et al., 2022). These studies are only designed for a single analysis and significantly require an intensive search process, thus having very limited generalizability with low efficiency.

Recently, numerous papers have proposed zero-shot NAS (Krishnakumar et al., 2022) to accelerate the search procedure substantially. The key idea of zero-shot NAS is to use proxies for estimating the final performance of neural networks *without training*, e.g., simply by summing the Euclidean norm of the gradients after a single minibatch, resulting in a dramatic improvement in the time efficiency (Abdelfattah et al., 2021). These proxies are usually theoretically grounded by analyzing the architecture properties (Tanaka et al., 2020; Lin et al., 2021) or gradient statistics (Mellor et al., 2021; Shu et al., 2022; Li et al., 2023). Since the first zero-shot proxy, there have been many follow-up techniques (Li et al., 2024), but *none* of them is designed for time-series analysis.

This paper aims to build a unified zero-shot NAS framework for general time-series analysis, capable of effectively designing neural architectures for a variety of tasks and input time series. Still, due to the distinct underlying characteristics of different time series and downstream tasks, recent surveys (Foumani et al., 2023; Wang et al., 2024) suggest that diverse neural architectures and configurations are necessary to achieve satisfactory performance across various analysis tasks. Several studies (Ning et al., 2021; Krishnakumar et al., 2022; White et al., 2022) also demonstrate that most zero-shot proxies do not generalize well and exhibit inconsistent performance across diverse datasets and downstream tasks. Thus, we aim to address the following research question.

How can we devise a zero-shot proxy that better correlates with the downstream performances across different time-series datasets and tasks?

To answer this question, we propose a novel zero-shot NAS framework for general time-series analysis with **Time-Frequency Aware Scoring**, named **TFAS**. Specifically, we propose a new time-frequency aware scoring method based on a *time-series properties extraction* module. This method extracts useful information from the input time series via both time- and frequency-domain decomposition. The extracted information is then combined with its corresponding target variables to compute the scores of candidate architectures during the search process. To incorporate **TFAS** into existing general-purpose time-series

models (e.g., TimesNet (Wu et al., 2023)), we introduce the concept of *augmented architectures*, which modify the models using an embedding module. As shown in Fig. 1, unlike existing methods that compute the proxy score solely based on the original input samples, our *TFAS*-based proxy incorporates time-series properties, resulting in more accurate estimations for the given neural architectures and datasets. Jointly incorporating time-series characteristics and task labels into the scoring procedure is crucial, as most zero-shot proxies exhibit unreliable performance across different datasets and tasks (White et al., 2022), even when computed using gradient information from input data. Consequently, *TFAS* improves the ranking reliability and consistency of the zero-shot proxy across diverse datasets and tasks. Our **main contributions** are summarized as follows.

- We propose *TFAS*, a novel zero-shot NAS framework that automatically discovers well-performing neural architectures for various time-series analysis tasks.
- We devise a new time-frequency aware scoring proxy that jointly leverages decomposed time-series properties and task labels to give a more accurate estimation of untrained candidate architectures.
- To verify the effectiveness of *TFAS*, we conduct extensive experiments on five downstream tasks: short-term forecasting, long-term forecasting, classification, anomaly detection, and imputation. The results show that the architectures discovered by *TFAS* outperform state-of-the-art handcrafted baselines by up to 23.6%.

2 Related work

2.1 Time-series analysis

Data mining tasks in time-series analysis typically focus on modeling temporal variations as a primary problem. Classical methods (Anderson and Kendall, 1976; Taylor and Letham, 2018) assume the presence of predefined patterns in temporal variations. However, the complexity of temporal variations in real-world time series often makes it difficult to capture these patterns adequately, limiting the applicability of such methods. To effectively model intricate temporal variations, several studies have proposed deep learning models based on various network architectures, such as multi-layer perceptrons (MLP), convolutional neural networks (CNN), recurrent neural networks (RNN), and Transformers.

MLP-based methods (Challu et al., 2023; Zeng et al., 2023; Wang et al., 2024) use MLP layers to encode the temporal dependencies into a fixed latent space along the temporal

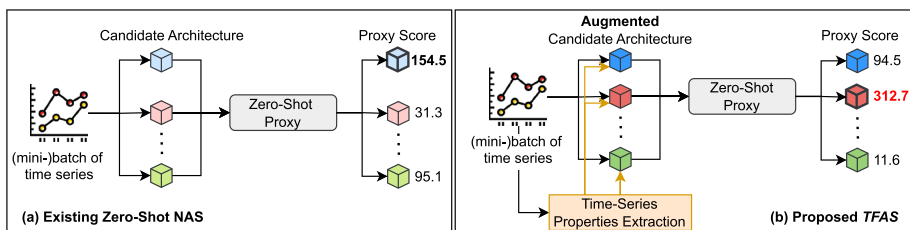


Fig. 1 Comparison of scoring procedures: **a** existing zero-shot NAS computes a proxy using only the original time series; **b** *TFAS* incorporates time-frequency properties into the proxy computation, resulting in a more accurate ranking

dimension. CNN-based models (Franceschi et al., 2019; Yue et al., 2022; Luo and Wang, 2024) learn the temporal variations using convolutional kernels sliding along the temporal dimension. Instead of standard 1D time-series modeling, Wu et al. (2023) explore temporal variations in a 2D space, enhancing the representation capability of time-series data. RNN-based methods (Lai et al., 2018; Nam et al., 2023; Gu et al., 2022) use a recurrent structure to capture temporal variations implicitly by state transitions between time steps. Lately, Transformers-based models (Zhou et al., 2021, 2022; Zhang and Yan, 2023) capture the temporal dependencies among time points with several designs of attention mechanisms. To further expand the receptive field, iTransformer (Liu et al., 2024) uniquely applies the attention mechanism to global representations of the entire series, capturing multivariate correlations at the series level. Extensive reviews can be found in the relevant surveys (Wang et al., 2024; Wen et al., 2023; Trirat et al., 2024).

2.2 Neural architecture search

According to recent surveys (White et al., 2023; Galván and Mooney, 2021), neural architecture search (NAS) has garnered increasing attention and popularity in recent years, thanks to advancements in hardware since its inception as neuroevolution (Tenorio and Lee, 1988). Typically, a NAS framework requires *three* core components. First, a *search space* is needed to define which architectures can be represented in principle. In this work, the search spaces are general-purpose time-series models, such as TimesNet (Wu et al., 2023) and ModernTCN (Luo and Wang, 2024), which can be adapted to various downstream tasks. Second, a *search strategy* is employed to explore the search space. Commonly used search strategies include reinforcement learning (Zoph and Le, 2017), evolutionary algorithms (Real et al., 2019), gradient-based methods (Liu et al., 2018), Bayesian optimization (White et al., 2021), prediction-based methods (White et al., 2021), and even random search (Li and Talwalkar, 2020). Finally, a *performance estimator* is required to evaluate candidate architectures on downstream tasks with unseen data. Early NAS frameworks (Real et al., 2019) evaluated candidate architectures by training from scratch, which incurred enormous computational costs, limiting their adoption in real-world environments. To address this issue, various approaches have been proposed, such as early stopping, weight sharing (Liu et al., 2018), and more recently, **zero-shot proxies** for performance estimation (Li et al., 2024), all of which significantly reduce the computational burden.

Zero-shot (or zero-cost) NAS aims to rank or estimate the performance of candidate neural architectures *without training*. Thus, we can bypass the expensive training process and discover many well-performing architectures using more computation-efficient proxies. Commonly, existing studies either extract the underlying properties of neural architectures (Tanaka et al., 2020; Abdelfattah et al., 2021; Lin et al., 2021; Jiang et al., 2023) or derive gradient statistics from a minibatch of data (Mellor et al., 2021; Zhang and Jia, 2022; Shu et al., 2022; Li et al., 2023). These zero-shot proxies are used to estimate the expressivity, complexity, or generalization capacity of neural networks expected to represent the final performance of the candidate architectures on a given downstream task. For example, Zhang and Jia (2022) propose GradSign, which approximates the optimization landscape of CNNs by evaluating the gradient signs for a mini-batch of samples. This approach assumes a nearly convex and semi-smooth landscape over a large neighborhood in a randomly initialized network for performance evaluation. Recently, ZiCo (Li et al., 2023) introduce an inverse coefficient of variation on the gradient to improve consistency.

This method considers the mean and standard deviation of gradients in each layer with a focus on convergence rate and network capacity.

Following the success of NAS in diverse tasks (Wang et al., 2020; Mo and Liu, 2021; Li et al., 2020; Pan et al., 2021), efforts have been made to automate the design of neural architectures for time-series analysis, focusing primarily on classification (Rakhshani et al., 2020; Xiao et al., 2021; Ren et al., 2022), forecasting (Wu et al., 2021; Deng et al., 2022; Lai et al., 2023; Wu et al., 2024), and anomaly detection (Haq et al., 2024; Trirat and Lee, 2024). These approaches are often limited to specific tasks and/or rely on computationally expensive search methods. In contrast, we propose a NAS framework for general time-series analysis that supports various downstream tasks using our newly proposed time-frequency aware zero-shot proxy combined with augmented architectures, significantly reducing search costs.

3 TFAS: time-frequency aware scoring

This section presents the problem formulation of zero-shot NAS for general time-series analysis and describes the details of the proposed *TFAS*, including the time-series properties extraction, architecture aggregation, and *TFAS*-based search. An overview of *TFAS* is illustrated in Fig. 2.

3.1 Problem definition

Let $\mathcal{X}$ denote a d -variate time series with observations $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ where $\mathbf{x}_t \in \mathbb{R}^d$ and $\mathcal{Y}$ denote target variables. The target variables can be a set of scalar values $\mathcal{Y} = (y_1, \dots, y_T)$, $y_t \in \mathbb{Z}$ (e.g., classification) or a set of d -dimensional vectors $\mathcal{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$, $\mathbf{y}_t \in \mathbb{R}^d$ (e.g., forecasting). Let $\mathcal{A} = \{A_i\}_{i=1}^N$ denote a set of N candidate architectures sampled from a search space $\mathcal{S}$. Each architecture A_i is defined by a specific configuration of network components (e.g., number of layers and channels). Finally, given a time-series analysis task, we estimate the architecture performance by using a zero-shot

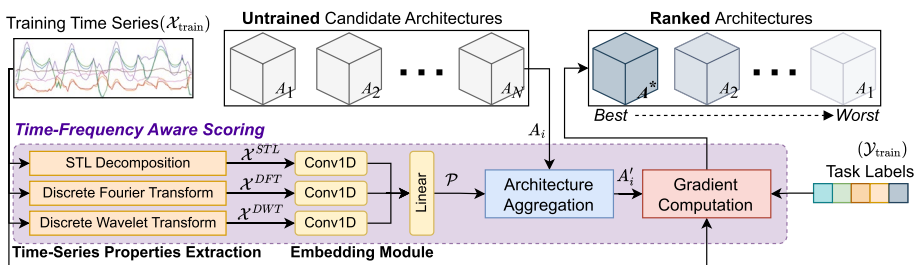


Fig. 2 Overall procedure of *TFAS*. Given a training time series $\mathcal{X}_{\text{train}}$, *TFAS* extracts the underlying characteristics ($\mathcal{X}^{\text{STL}}$, $\mathcal{X}^{\text{DFT}}$, $\mathcal{X}^{\text{DWT}}$) of the input using the time-series properties extraction module. During the search, *TFAS* gets all the extracted properties through the embedding module $\mathcal{P}$ attached to each candidate architecture A_i via architecture aggregation, along with the original time series and target labels $\mathcal{Y}_{\text{train}}$, to compute the architecture scores for the final ranking

proxy $\mathbf{Q} = \mathcal{Q}(A, \mathcal{X}, \mathcal{Y})$. $\mathcal{Q}$ can be as simple as #Params or FLOPs. In general, higher $\mathbf{Q}$ indicates better architectures.

Zero-Shot NAS for Time-Series Analysis: *Given a training time series with its labels $\{\mathcal{X}_{\text{train}}, \mathcal{Y}_{\text{train}}\}$ and a set of **untrained** architectures $\mathcal{A}$, select the architecture A^* that maximizes the proxy $\mathcal{Q}$. Formally, we solve*

$$A^* \in \arg \max_{A \in \mathcal{A}} \mathcal{Q}(A, \mathcal{X}_{\text{train}}, \mathcal{Y}_{\text{train}}). \quad (1)$$

3.2 Time-series properties extraction

Except the raw input time series, the zero-shot proxy does not know about the characteristics of the time series because it *cannot* learn. Thus, we devise a time-series properties extraction module based on the following techniques to jointly inform the proxy about the underlying characteristics of the current dataset and task.

3.2.1 Time-domain decomposition

To uncover the underlying patterns in time series data, we employ Seasonal and Trend decomposition using Loess (STL) decomposition (Cleveland et al., 1990) to decompose time series in the time domain. STL decomposition is a robust method for analyzing the underlying components of a time series by separating it into three parts: seasonal, trend, and residual. This approach is particularly useful for analyzing temporal patterns, identifying cyclical behaviors, and isolating irregular variations. Its flexibility in handling non-linear trends and varying seasonal patterns makes it valuable for representing time series behaviors. Formally, $\mathcal{X}^{\text{STL}} = \text{STL}(\mathcal{X})$.

3.2.2 Frequency-domain decomposition

Recently, it has been proven that frequency-domain information is vital for time-series analysis (Yi et al., 2023). We accordingly adopt frequency-domain transformations to extract useful information that can decompose and represent global characteristics of time series in the frequency domain. Specifically, we use discrete Fourier transform (Zhang et al., 2022), denoted as $\mathcal{X}^{\text{DFT}} = \text{DFT}(\mathcal{X})$, along with discrete wavelet transform (Zhang et al., 2022), denoted as $\mathcal{X}^{\text{DWT}} = \text{DWT}(\mathcal{X})$, to obtain both global-view properties and multi-scale representations of the given time-series data.

In this paper, we chose STL, DFT, and DWT due to their complementary coverage of time-series characteristics and ease of integration. Our selection was motivated by three considerations: (1) broad coverage of both time and frequency domains, (2) demonstrated utility in prior time-series tasks (as evidenced in TimesNet (Wu et al., 2023) and FED-former (Zhou et al., 2022)), and (3) computational efficiency for zero-shot NAS. Note that other decomposition methods, e.g., SSA and EMD, could offer complementary signals, yet they often require iterative empirical mode estimation or eigen-decomposition, which significantly increases computational burden. In addition, SSA and EMD might capture

similar information to our chosen techniques—SSA can extract principal components similar to frequency components; EMD provides adaptive frequency bands similar to wavelets.

3.3 Embedding and architecture aggregation

Before the search phase, we create an embedding module $\mathcal{P}$ consisting of three one-dimensional convolutional layers (Conv1D) corresponding to each set of time-series properties and a linear layer (Linear). As in Eq. (2) below, the time-series properties are embedded via Conv1D layers and concatenated via the final Linear layer.

$$\mathcal{P} = \text{Linear}([\text{Conv1D}_{STL}(\mathcal{X}^{STL}); \text{Conv1D}_{DFT}(\mathcal{X}^{DFT}); \text{Conv1D}_{DWT}(\mathcal{X}^{DWT})]), \quad (2)$$

where “;” indicates the concatenate operation. As we aim to evaluate whether the extracted time-series properties can directly provide useful information to the zero-shot proxy, we employ convolutional and linear layers in our embedding module. This approach aligns with the original scoring capability of the base zero-shot proxy (Li et al., 2023), thereby mitigating the influence of architectural design choices.

During the search phase, we aggregate the embedding module $\mathcal{P}$ on top of each candidate architecture A_i to pass all the extracted information as auxiliary inputs through the candidate architecture. As a result, we transform each candidate network into an *augmented* network $A'_i = A_i(\mathcal{P})$.

3.4 Search with time-frequency aware scoring

This paper employs gradient statistics based on a single forward pass on training mini-batches to compute the scores for candidate architectures. In particular, we use gradient mean and standard deviation, proven efficient, as the base zero-shot proxy (Li et al., 2023). These statistics quantify the stability of gradients across mini-batches. A higher ratio indicates more stable gradient updates, making it a reliable proxy for downstream performance without requiring extensive training. Unlike the previous work, we compute the score with the augmented neural architectures A'_i based on the above time-series properties extraction and embedding modules instead of calculating the score on the bare neural architectures. Hence, the proxy can directly reflect both the original input time series and underlying characteristics when scoring the candidate architectures (as depicted in Fig. 1b). Formally, given L trainable layers in an augmented neural architecture A'_i and loss function $\mathcal{L}$, the *time-frequency aware* zero-shot proxy $\mathcal{Q}$ is defined by

$$\mathcal{Q} = \sum_{l=1}^L \log \left(\sum_{\omega \in \theta_l} \frac{\mathbb{E}[|\nabla_{\omega} \mathcal{L}(\mathcal{X}'_b, \mathcal{Y}_b; \Theta)|]}{\text{STD}[|\nabla_{\omega} \mathcal{L}(\mathcal{X}'_b, \mathcal{Y}_b; \Theta)|]} \right), \quad (3)$$

where $b \in \{1, \dots, B\}$ and B is the number of training batches used to compute the scores $\mathcal{Q}$. Θ denotes the initial parameters of the given augmented architecture A'_i , θ_l denotes the parameters of the l -th layer of the network, ω represents each element in θ_l , $\mathcal{X}'_b$ and $\mathcal{Y}_b$ are the b -th input batch and corresponding labels from the training set. $\mathcal{X}'_b = [\mathcal{X}_b; \mathcal{X}_b^{STL}; \mathcal{X}_b^{DFT}; \mathcal{X}_b^{DWT}]$. $\log$ is used to stabilize the computation by regularizing the extremely large or small values (Li et al., 2023).

In this paper, we use random search for conducting zero-shot NAS due to its concise and straightforward implementation, yet it is strong enough to achieve state-of-the-art

performance on downstream tasks (Bergstra and Bengio, 2012; Li and Talwalkar, 2020; Lindauer and Hutter, 2020). As outlined in Algorithm 1, we search for the neural architecture A^* with the highest $\mathbf{Q}$ score from a search space $\mathcal{S}$, given N candidate architectures. For each architecture $A_i \in \mathcal{A}$, we compute the score $\mathbf{Q}_i$ with the augmented version A'_i and save each computed score for the final ranking. After calculating the scores for all candidates, we select the architecture with the highest score as the final architecture. Note that the values in Eq. (3) are computed with the initial parameters, i.e., it never updates the parameters when computing the score.

Algorithm 1 *TFAS* with random search.

Input: search space $\mathcal{S}$, number of candidates N , batches of input and extracted time series $\mathbb{D} = \{(\mathcal{X}'_b, \mathcal{Y}_b)\}_{b=1}^B$
Output: optimal architecture A^*
Initialize set of candidate architecture scores $\mathbb{Q} = \emptyset$
Randomly sample N candidate architectures $\mathcal{A}$ from $\mathcal{S}$
for $i = 1$ **to** N **do**
 Create augmented network A'_i of $A_i \in \mathcal{A}$ by Eq. (2)
 Compute score $\mathbf{Q}_i$ for A'_i on $\mathbb{D}$ using Eq. (3)
 Add $\mathbf{Q}_i$ to $\mathbb{Q}$
end for
 $A^* =$ the candidate network A_i with the highest $\mathbf{Q}_i \in \mathbb{Q}$
return A^*

4 Experiments

To verify *TFAS*'s effectiveness, we extensively experiment on five downstream tasks (including short- and long-term forecasting, classification, anomaly detection, and imputation) as well as ranking correlation analysis and ablation study. The source code is available at <https://github.com/kaist-dmlab/TFAS>.

Table 1 Summary of benchmark data and evaluation metrics

Downstream tasks	Benchmarks	Evaluation metrics	Series lengths
Forecasting	Short-term: M4 (6 subsets)	SMAPE, MASE, OWA	6–48
	Long-term: ETT (4 subsets), Electricity, Traffic, Weather, Exchange	MSE, MAE	96–720
Classification	UEA (10 subsets)	Accuracy	29–1751
Anomaly Detection	SMD, MSL, SMAP, SWaT, PSM	Precision, Recall, F1	100
Imputation	ETT (4 subsets), Electricity, Weather	MSE, MAE	96

Table 2 Training configurations of each task used for experiments, following Wu et al. (2023)

Downstream tasks	LR	Loss ($\mathcal{L}$)	Batch size	# Epoch
Short-term forecasting	10^{-3}	SMAPE	16	10
Long-term forecasting	10^{-4}	MSE	32	10
Classification	10^{-3}	Cross Entropy	16	30
Anomaly detection	10^{-4}	MSE	128	10
Imputation	10^{-3}	MSE	16	10

We use the Adam optimizer with the default hyperparameters for (β_1, β_2) as (0.9, 0.999). LR means the initial learning rate

4.1 Experimental setup

4.1.1 Implementation details

We adhere to the *exact* experimental setups and implementation details as in the baseline study using TSLib library (Wu et al., 2023), including benchmark datasets and evaluation metrics (Table 1) and training configurations (Table 2) used for each task, to ensure a straightforward and fair comparison with prior studies. Specifically, by fixing the downstream task hyperparameters, we can evaluate whether the search method actually identifies well-performing models under predefined setups.

The experiments are conducted on a single NVIDIA GeForce RTX 3090 GPU. For the zero-shot proxy hyperparameters, we set the number of input batches to two ($B = 2$) as suggested by Li et al. (2023), making the score computation very time efficient. The number of candidate architectures N is 1000. To evaluate the effectiveness of *TFAS* across diverse downstream tasks, we use TimesNet (Wu et al., 2023) as our search space $\mathcal{S}$. Specifically, we search for the number of top- k amplitudes, the number of layers, dropout ratios, embedding methods, deep feature dimensions, output channel expansion ratios, and the number of kernels.

For each task and dataset, we sample $N = 1000$ architectures once (using a fixed random seed for the random sampler) by randomly sampling from the search space $\mathcal{S}$ and evaluate them with the zero-shot proxy to select the best architecture. Each hyperparameter of the architecture (e.g., layer count and kernel size) is sampled uniformly from a predetermined range or set of values. This random sampling ensures a diverse set of architectures without bias.

4.1.2 Evaluation metrics

We adopt the symmetric mean absolute percentage error (SMAPE), mean absolute scaled error (MASE) and overall weighted average (OWA) as the metrics, following Oreshkin et al. (2020), for the short-term forecasting. For the long-term forecasting and imputation tasks, we use the mean square error (MSE) and mean absolute error (MAE) metrics. Last, we adopt accuracy metric for classification, while the precision, recall, and F1 scores are used for anomaly detection.

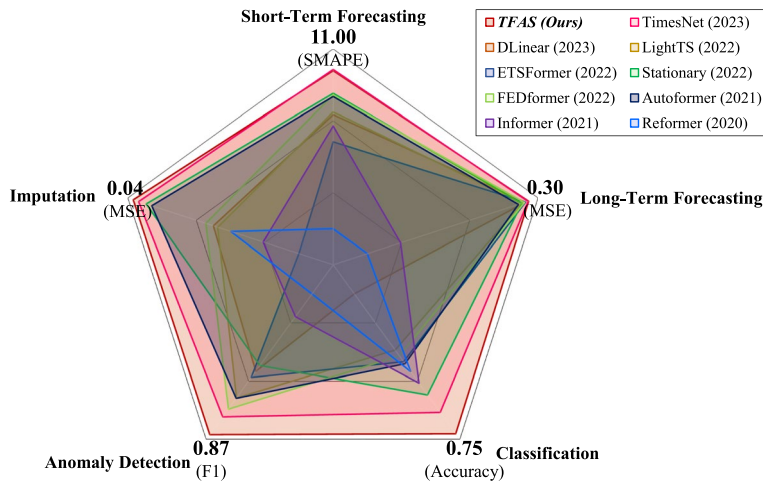


Fig. 3 Overall performance comparison showing that *TFAS* outperforms most baselines across five downstream tasks, particularly in anomaly detection and classification

Table 3 Summary of search costs across different downstream tasks ($N = 1000$)

Downstream tasks	# Datasets	Total cost (GPU Hours)	Cost/dataset (GPU Hours)	Cost/arch (GPU hours)
Short-Term Forecasting	6	0.72	0.12	0.0001
Long-Term Forecasting	8	67.67	8.46	0.0085
Classification	10	136.14	13.61	0.0136
Anomaly Detection	5	2.89	0.58	0.0006
Imputation	6	12.98	2.16	0.0022

4.1.3 Baseline methods

In addition to TimesNet (Wu et al., 2023), we extensively compare the following well-known and advanced models in all five analysis tasks. **RNNs:** LSTM (Hochreiter and Schmidhuber, 1997), LSTNet (Lai et al., 2018), and LSSL (Gu et al., 2022); **CNNs:** TCN (Franceschi et al., 2019); **MLPs:** LightTS (Zhang et al., 2022) and DLinear (Zeng et al., 2023); **Transformers:** Reformer (Kitaev et al., 2020), Informer (Zhou et al., 2021), Pyraformer (Liu et al., 2022), Autoformer (Wu et al., 2021), FEDformer (Zhou et al., 2022), Non-stationary Transformers (Liu et al., 2022), and ETSformer (Woo et al., 2022). Besides, we also compare the **task-specific state-of-the-art models**, such as N-HiTS (Challu et al., 2023) and N-BEATS (Oreshkin et al., 2020) for short-term forecasting, Anomaly Transformer (Xu et al., 2022) for anomaly detection, Rocket (Dempster et al., 2020) and Flowformer (Wu et al., 2022) for classification, and so on.

4.2 Main results

Overall, *TFAS* mostly achieves state-of-the-art performance in five downstream tasks compared with other customized models (Fig. 3), given the search cost ranging from about 7 GPU minutes to 14 GPU hours for each dataset. Although *TFAS* may require up to about 14 GPU hours for a single dataset in our tests (e.g., classification tasks), this is orders of magnitude faster than a full neural architecture search that trains each candidate model (which could easily exceed 10× or 100× more compute). Note that the search cost depends on batch and dataset sizes set differently for each task. Table 3 presents the total and average search costs for all tasks. To demonstrate the reliability of our results, we include additional runs and report the results in §A, comparing our method with the second-best TimesNet. Specifically, we repeat the search process exactly three times and report the mean and standard deviation of the performance across runs. Experimental setups and results of each downstream task are presented below (Table 4).

4.2.1 Short-term forecasting

In this experiment, we use the M4 (Makridakis, 2018) containing the yearly, quarterly and monthly collected univariate marketing data. M4 involves 100K different time series collected in different frequencies.

Results. As presented in Table 5, *TFAS* demonstrates great performance in the short-term forecasting task. As the M4 dataset is collected from different sources, the temporal variations can be quite diverse, making this task much more challenging. Despite the time used for the search of 1,000 architectures being only about 43.2 GPU minutes, the models found by *TFAS* still perform on par with the state-of-the-art models that require a massive amount of manual trial-and-error and exhaustive hyperparameter tuning.

4.2.2 Long-term forecasting

For the long-term forecasting task, we use ETT (Zhou et al., 2021), Electricity (UCI, 2015), Traffic (Cuturi, 2011), Weather (Wetterstation, 2020), and Exchange (Lai et al., 2018), covering four real-world applications. As each dataset contains only one continuous time series, we obtain samples by a sliding window (Wu et al., 2023).

Results. Interestingly, for long-term forecasting, *TFAS* achieves state-of-the-art performance on several datasets and prediction lengths using the search cost of only 30.6 GPU seconds per architecture, which is negligible compared to the standard training with manual trial-and-error, considering this performance improvement. Table 6 presents the experimental results of the long-term forecasting task.

4.2.3 Classification

We adopt the sequence-level classification to verify the model capacity in high-level representation learning and preprocess the time-series data such that different subsets have different sequence lengths. Following Wu et al. (2023), we use *ten* multivariate datasets from the UEA archive (Bagnall et al., 2018), including heartbeat monitoring, audio, gesture, and action recognition, along with other practical applications.

Results. As shown in Table 7, *TFAS* achieves the state-of-the-art performance with an average accuracy of 74.7%, surpassing the previous state-of-the-art foundation model

Table 4 Full results for the **anomaly detection** task. **P**, **R**, and **F1** represent the precision, recall, and F1 score (%), respectively

Datasets metrics	SMD			MSL			SMAP			SWaT			PSM			Avg	
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	F1
LSTM	78.52	65.47	71.41	78.04	86.22	81.93	91.06	57.49	70.48	78.06	91.72	84.34	69.24	99.53	81.67	77.97	77.97
Transformer	83.58	76.13	79.56	71.57	87.37	78.68	89.37	57.12	69.70	68.84	96.53	80.37	62.75	96.56	76.07	76.88	76.88
LogTrans	83.46	70.13	76.21	73.05	87.37	79.57	89.15	57.59	69.97	68.67	97.32	80.52	63.06	98.00	76.74	76.60	76.60
TCN	84.06	79.07	81.49	75.11	82.44	78.60	86.90	59.23	70.45	76.59	95.71	85.09	54.59	99.77	70.57	77.24	77.24
Reformer	82.58	69.24	75.32	85.51	83.31	84.40	90.91	57.44	70.40	72.50	96.53	82.80	59.93	95.38	73.61	77.31	77.31
Informer	86.60	77.23	81.65	81.77	86.48	84.06	90.11	57.13	69.92	70.29	96.75	81.43	64.27	96.33	77.10	78.83	78.83
Anom. Trans	88.91	82.23	<u>85.49</u>	79.61	87.37	83.31	91.85	58.11	71.18	72.51	97.32	83.10	68.35	94.72	79.40	80.50	80.50
Pyraformer	85.61	80.61	83.04	83.81	85.93	<u>84.86</u>	92.54	57.71	71.09	87.92	96.00	91.78	71.67	96.02	82.08	82.57	82.57
Autoformer	88.06	82.35	85.11	77.27	80.92	79.05	90.40	58.62	71.12	89.85	95.81	92.74	99.08	88.15	93.29	84.26	84.26
LSSL	78.51	65.32	71.31	77.55	88.18	82.53	89.43	53.43	66.90	79.05	93.72	85.76	66.02	92.93	77.20	76.74	76.74
Stationary	88.33	81.21	84.62	68.55	89.14	77.50	89.37	59.02	71.09	68.03	96.75	79.88	97.82	96.76	<u>97.29</u>	82.08	82.08
DLinear	83.62	71.52	77.10	84.34	85.42	84.88	92.32	55.41	69.26	80.91	95.30	87.52	98.28	89.26	93.55	82.46	82.46
ETSformer	87.44	79.23	83.13	85.13	84.93	85.03	92.25	55.75	69.50	90.02	80.36	84.91	99.31	85.28	91.76	82.87	82.87
LightTS	87.10	78.42	82.53	82.40	75.78	78.95	92.58	55.27	69.21	91.98	94.72	93.33	98.37	95.97	97.15	84.23	84.23
FEDformer	87.95	82.39	85.08	77.14	80.07	78.57	90.47	58.10	70.76	90.17	96.42	<u>93.19</u>	97.31	97.16	97.23	84.97	84.97
TimesNet	87.76	82.63	85.12	82.97	85.42	84.18	91.50	57.80	70.85	88.31	96.24	92.10	98.22	92.21	95.21	<u>85.49</u>	<u>85.49</u>
TFAS	88.78	82.64	85.60	81.57	85.51	83.49	90.41	61.93	73.50	91.45	95.29	93.33	98.50	96.39	97.43	86.67	86.67

Bold and underlined text indicates the best and second-best scores, respectively

Table 5 Full results for the **short-term forecasting** task. “.” in the Transformers indicates the name of *former (e.g., “Re.” means Reformer, “Auto.” means Autoformer”, and “FED.” means FEDformer”). Stationary means the Non-stationary Transformer

Models metrics	LSSL	TCN	LSTM	Re	LogTrans	In	Pyra	Auto	Station-ary	FED	DLin-ear	LightTS	ETS	N-BEATS	N-HITS	TimesNet	TFAS	
Yearly	SMAPE	61.675	14.920	176.040	16.169	17.107	14.727	15.530	13.974	13.717	13.728	16.965	14.247	18.009	13.436	13.418	<u>13.387</u>	13.363
	MASE	19.953	3.364	31.033	3.800	4.177	3.418	3.711	3.134	3.078	3.048	4.283	3.109	4.487	3.043	3.045	2.996	<u>3.012</u>
	OWA	4.397	0.880	9.290	0.973	1.049	0.881	0.942	0.822	0.807	0.803	1.058	0.827	1.115	0.794	0.793	0.786	<u>0.789</u>
Quarterly	SMAPE	65.999	11.122	172.808	13.313	13.207	11.360	15.449	11.338	10.958	10.792	12.145	11.364	13.376	10.124	10.202	<u>10.100</u>	10.087
	MASE	17.662	1.360	19.753	1.775	1.827	1.401	2.350	1.365	1.325	1.283	1.520	1.328	1.906	1.169	1.194	1.182	<u>1.180</u>
	OWA	9.436	1.001	15.049	1.252	1.266	1.027	1.558	1.012	0.981	0.958	1.106	1.000	1.302	0.886	0.899	0.890	<u>0.889</u>
Monthly	SMAPE	64.664	15.626	143.237	20.128	16.149	14.062	17.642	13.958	13.917	14.260	13.514	14.014	14.588	<u>12.677</u>	12.791	12.670	12.755
	MASE	16.245	1.274	16.551	2.614	1.660	1.141	1.913	1.103	1.097	1.102	1.037	1.053	1.368	<u>0.937</u>	0.969	0.933	0.939
	OWA	9.879	1.141	12.747	1.927	1.340	1.024	1.511	1.002	0.998	1.012	0.956	0.981	1.149	<u>0.880</u>	0.899	0.878	0.884
Others	SMAPE	121.844	7.186	186.282	32.491	23.236	24.460	24.786	5.485	6.302	4.954	6.709	15.880	7.267	<u>4.925</u>	5.061	4.891	4.935
	MASE	91.650	4.677	119.294	33.355	16.288	20.960	18.581	3.865	4.064	3.264	4.953	11.434	5.240	3.391	3.216	<u>3.302</u>	3.318
	OWA	27.273	1.494	38.411	8.679	5.013	5.879	5.538	1.187	1.304	1.036	1.487	3.474	1.591	1.053	<u>1.040</u>	1.035	1.043
Weighted Average	SMAPE	67.156	13.961	160.031	18.200	16.018	14.086	16.987	12.909	12.780	12.840	13.639	13.525	14.718	<u>11.851</u>	11.927	11.829	11.876
	MASE	21.208	1.945	25.788	4.223	3.010	2.718	3.265	1.771	1.756	1.701	2.095	2.111	2.408	1.599	1.613	1.585	<u>1.598</u>
	OWA	8.021	1.023	12.642	1.775	1.378	1.230	1.480	0.939	0.930	0.918	1.051	1.051	1.172	<u>0.855</u>	0.861	0.851	0.857

Bold and underlined text indicates the best and second-best scores, respectively

Table 6 Full results for the **long-term forecasting** task. We compare extensive competitive models under different prediction lengths [96, 192, 336, 720]. The input sequence length is set to 96

Models	LSTM		LSSL		Reformer		LogTrans		Informer		Pyraformer		Autoformer		Stationary	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	0.863	0.664	0.450	0.477	0.538	0.528	0.600	0.546	0.672	0.571	0.543	0.510	0.505	0.475	0.398
	192	1.113	0.776	0.469	0.481	0.658	0.592	0.837	0.700	0.795	0.669	0.557	0.537	0.553	0.496	0.444
	336	1.267	0.832	0.583	0.574	0.898	0.721	1.124	0.832	1.212	0.871	0.754	0.655	0.621	0.537	0.464
	720	1.324	0.858	0.632	0.596	1.102	0.841	1.153	0.820	1.166	0.823	0.908	0.724	0.671	0.561	0.516
	Avg	1.142	0.783	0.534	0.532	0.799	0.671	0.929	0.725	0.961	0.734	0.691	0.607	0.588	0.517	0.456
ETTm2	96	2.041	1.073	0.243	0.342	0.658	0.619	0.768	0.642	0.365	0.453	0.435	0.507	0.255	0.339	0.274
	192	2.249	1.112	0.392	0.448	1.078	0.827	0.989	0.757	0.533	0.563	0.730	0.673	0.281	0.340	0.339
	336	2.568	1.238	0.932	0.724	1.549	0.972	1.334	0.872	1.363	0.887	1.201	0.845	0.339	0.372	0.361
	720	2.720	1.287	1.372	0.879	2.631	1.242	3.048	1.328	3.379	1.338	3.625	1.451	0.433	0.432	0.413
	Avg	2.395	1.178	0.735	0.598	1.479	0.915	1.535	0.900	1.410	0.810	1.498	0.869	0.327	0.371	0.347
ETTh1	96	1.044	0.773	0.548	0.528	0.837	0.728	0.878	0.740	0.865	0.713	0.664	0.612	0.449	0.459	0.491
	192	1.217	0.832	0.542	0.526	0.923	0.766	1.037	0.824	1.008	0.792	0.790	0.681	0.500	0.482	0.504
	336	1.259	0.841	1.298	0.942	1.097	0.835	1.238	0.932	1.107	0.809	0.891	0.738	0.521	0.496	0.535
	720	1.271	0.838	0.721	0.659	1.257	0.889	1.135	0.852	1.181	0.865	0.963	0.782	0.514	0.512	0.616
	Avg	1.198	0.821	0.777	0.664	1.029	0.805	1.072	0.837	1.040	0.795	0.827	0.703	0.496	0.487	0.537
ETTh2	96	2.522	1.278	1.616	1.036	2.626	1.317	2.116	1.197	3.755	1.525	0.645	0.597	0.346	0.388	0.458
	192	3.312	1.384	2.083	1.197	11.120	2.979	4.315	1.635	5.602	1.931	0.788	0.683	0.456	0.452	0.493
	336	3.291	1.388	2.970	1.439	9.323	2.769	1.124	1.604	4.721	1.835	0.907	0.747	0.482	0.486	0.551
	720	3.257	1.357	2.576	1.363	3.874	1.697	3.188	1.540	3.647	1.625	0.963	0.783	0.515	0.511	0.560
	Avg	3.096	1.352	2.311	1.259	6.736	2.191	2.686	1.494	4.431	1.729	0.826	0.703	0.450	0.459	0.516
Electricity	96	0.375	0.437	0.300	0.392	0.312	0.402	0.258	0.357	0.274	0.368	0.386	0.449	0.201	0.317	0.273
	192	0.442	0.473	0.297	0.390	0.348	0.433	0.266	0.368	0.296	0.386	0.378	0.443	0.222	0.334	0.286
	336	0.439	0.473	0.317	0.403	0.350	0.433	0.280	0.380	0.300	0.394	0.376	0.443	0.231	0.338	0.304
	720	0.980	0.814	0.338	0.417	0.340	0.420	0.283	0.376	0.373	0.439	0.376	0.445	0.254	0.361	0.321
	Avg	0.559	0.549	0.313	0.401	0.338	0.422	0.272	0.370	0.311	0.397	0.379	0.445	0.227	0.338	0.296

Table 6 (continued)

Models		LSTM		LSSL		Reformer		LogTrans		Informer		Pyraformer		Autoformer		Stationary	
Metrics		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Traffic	96	0.843	0.453	0.798	0.436	0.732	0.423	0.684	0.384	0.719	0.391	0.867	0.468	0.613	0.388	0.612	0.338
	192	0.847	0.453	0.849	0.481	0.733	0.420	0.685	0.390	0.696	0.379	0.869	0.467	0.616	0.382	0.613	0.340
	336	0.853	0.455	0.828	0.476	0.742	0.420	0.734	0.408	0.777	0.420	0.881	0.469	0.622	0.337	0.618	0.328
	720	1.500	0.805	0.854	0.489	0.755	0.423	0.717	0.396	0.864	0.472	0.896	0.473	0.660	0.408	0.653	0.355
	Avg	1.011	0.542	0.832	0.471	0.741	0.422	0.705	0.395	0.764	0.416	0.878	0.469	0.628	0.379	0.624	0.340
Weather	96	0.369	0.406	0.174	0.252	0.689	0.596	0.458	0.490	0.300	0.384	0.622	0.556	0.266	0.336	0.173	0.223
	192	0.416	0.435	0.238	0.313	0.752	0.638	0.658	0.589	0.598	0.544	0.739	0.624	0.307	0.367	0.245	0.285
	336	0.455	0.454	0.287	0.355	0.639	0.596	0.797	0.652	0.578	0.523	1.004	0.753	0.359	0.395	0.321	0.338
	720	0.535	0.520	0.384	0.415	1.130	0.792	0.869	0.675	1.059	0.741	1.420	0.934	0.419	0.428	0.414	0.410
	Avg	0.444	0.454	0.271	0.334	0.803	0.656	0.696	0.602	0.634	0.548	0.946	0.717	0.338	0.382	0.288	0.314
Exchange	96	1.453	1.049	0.395	0.474	1.065	0.829	0.968	0.812	0.847	0.752	1.748	1.105	0.197	0.323	0.111	0.237
	192	1.846	1.179	0.776	0.698	1.188	0.906	1.040	0.851	1.204	0.895	1.874	1.151	0.300	0.369	0.219	0.335
	336	2.136	1.231	1.029	0.797	1.357	0.976	1.659	1.081	1.672	1.036	1.943	1.172	0.509	0.524	0.421	0.476
	720	2.984	1.427	2.283	1.222	1.510	1.016	1.941	1.127	2.478	1.310	2.085	1.206	1.447	0.941	1.092	0.769
	Avg	2.105	1.222	1.121	0.798	1.280	0.932	1.402	0.968	1.550	0.998	1.913	1.159	0.613	0.539	0.461	0.454
1st Count	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
Models	FEDformer		DLinear		LightTS		ETSformer		TimesNet		TFAS						
Metrics		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	0.379	0.419	0.345	0.372	0.374	0.400	0.375	0.398	0.338	0.375	0.319	0.366				
	192	0.426	0.441	0.380	0.389	0.400	0.407	0.408	0.410	0.374	0.387	0.381	0.396				
	336	0.445	0.459	0.413	0.413	0.438	0.438	0.435	0.428	0.410	0.411	0.407	0.418				
	720	0.543	0.490	0.474	0.453	0.527	0.502	0.499	0.462	0.478	0.450	0.496	0.462				
	Avg	0.448	0.452	0.403	0.407	0.435	0.437	0.429	0.425	0.400	0.406	0.401	0.411				

Table 6 (continued)

Models	Metrics	FEDformer				DLinear				LightTS				ETSformer				TimesNet				TFAS			
		MSE	MAE			MSE	MAE			MSE	MAE			MSE	MAE			MSE	MAE			MSE	MAE		
ETTm2	96	0.203	0.287			0.193	0.292			0.209	0.308			0.189	0.280			0.187	0.267			0.181	0.263		
	192	0.269	0.328			0.284	0.362			0.311	0.382			0.253	0.319			0.249	0.309			0.253	0.308		
	336	0.325	0.366			0.369	0.427			0.442	0.466			0.314	0.357			0.321	0.351			0.314	0.345		
	720	0.421	0.415			0.554	0.522			0.675	0.587			0.414	0.413			0.408	0.403			0.443	0.417		
	Avg	0.305	0.349			0.350	0.401			0.409	0.436			0.293	0.342			0.291	0.333			0.298	0.333		
ETTh1	96	0.376	0.419			0.386	0.400			0.424	0.432			0.494	0.479			0.384	0.402			0.395	0.413		
	192	0.420	0.448			0.437	0.432			0.475	0.462			0.538	0.504			0.436	0.429			0.437	0.444		
	336	0.459	0.465			0.481	0.459			0.518	0.488			0.574	0.521			0.491	0.469			0.483	0.466		
	720	0.506	0.507			0.519	0.516			0.547	0.533			0.562	0.535			0.521	0.500			0.500	0.486		
	Avg	0.440	0.460			0.456	0.452			0.491	0.479			0.542	0.510			0.458	0.450			0.454	0.452		
ETTh2	96	0.358	0.397			0.333	0.387			0.397	0.437			0.340	0.391			0.340	0.374			0.327	0.366		
	192	0.429	0.439			0.477	0.476			0.520	0.504			0.430	0.439			0.402	0.414			0.399	0.406		
	336	0.496	0.487			0.594	0.541			0.626	0.559			0.485	0.479			0.452	0.452			0.435	0.440		
	720	0.463	0.474			0.831	0.657			0.863	0.672			0.500	0.497			0.462	0.468			0.445	0.451		
	Avg	0.437	0.449			0.559	0.515			0.602	0.543			0.439	0.452			0.414	0.427			0.402	0.416		
Electricity	96	0.193	0.308			0.197	0.282			0.207	0.307			0.187	0.304			0.168	0.272			0.171	0.274		
	192	0.201	0.315			0.196	0.285			0.213	0.316			0.199	0.315			0.184	0.289			0.186	0.288		
	336	0.214	0.329			0.209	0.301			0.230	0.333			0.212	0.329			0.198	0.300			0.198	0.298		
	720	0.246	0.355			0.245	0.333			0.265	0.360			0.233	0.345			0.220	0.320			0.240	0.330		
	Avg	0.214	0.327			0.212	0.300			0.229	0.329			0.208	0.323			0.193	0.295			0.199	0.298		

Table 6 (continued)

Models	FEDformer				DLinear				LightTS				ETSformer				TimesNet				TFAS			
	MSE		MAE		MSE		MAE		MSE		MAE		MSE		MAE		MSE		MAE		MSE		MAE	
Traffic	96		<u>0.587</u>		0.650		0.396		0.615		0.391		0.607		0.392		0.593		<u>0.321</u>		0.580		0.320	
	192		0.604		<u>0.598</u>		0.370		0.601		0.382		0.621		0.399		0.617		<u>0.336</u>		0.583		0.321	
	336		0.621		<u>0.605</u>		0.373		0.613		0.386		0.622		0.396		0.629		0.336		0.598		0.327	
	720		0.626		0.645		0.394		0.658		0.407		<u>0.632</u>		0.396		0.640		<u>0.350</u>		0.645		0.347	
	Avg		<u>0.610</u>		0.625		0.383		0.622		0.392		0.621		0.396		0.620		<u>0.336</u>		0.602		0.329	
Weather	96		0.217		0.196		0.255		0.182		0.242		0.197		0.281		<u>0.172</u>		<u>0.220</u>		0.166		0.216	
	192		0.276		0.237		0.296		<u>0.227</u>		<u>0.287</u>		0.237		0.312		0.219		0.261		0.219		0.261	
	336		0.339		0.283		0.335		0.282		0.334		0.298		0.353		<u>0.280</u>		<u>0.306</u>		0.278		0.297	
	720		0.403		0.345		0.381		<u>0.352</u>		0.386		0.352		0.288		0.365		<u>0.359</u>		0.359		0.352	
	Avg		0.309		0.265		0.317		0.261		0.312		0.271		0.309		<u>0.259</u>		<u>0.287</u>		0.256		0.282	
Exchange	96		0.148		<u>0.088</u>		<u>0.218</u>		0.116		0.262		0.085		0.204		0.107		0.234		0.105		0.232	
	192		0.271		0.176		<u>0.315</u>		0.215		0.359		<u>0.182</u>		0.303		0.226		0.344		0.199		0.322	
	336		0.460		0.313		0.427		0.377		0.466		<u>0.348</u>		<u>0.428</u>		0.367		0.448		0.376		0.451	
	720		1.195		<u>0.839</u>		0.695		0.831		<u>0.699</u>		1.025		0.774		0.964		0.746		0.947		0.743	
	Avg		0.519		0.354		0.414		<u>0.385</u>		0.447		0.410		<u>0.427</u>		0.416		0.443		0.407		0.437	
1st Count	4		9		1		4		15		34		4		15		34		15		34		34	

Bold and underlined text indicates the best and second-best scores, respectively

Table 7 Full results for the **classification** task. “.” in the Transformers indicates the name of *former (e.g., “Re.” means Reformer, “Auto.” means Autoformer”, and “FED.” means FEDformer”). We report the classification accuracy (%) as the result

Models / Datasets	Classical Methods			RNNs		LST-Net		CNNs		Transformers			MLPs			TimesNet	TFAS					
	DTW		XGBoost	Rocket	LSTM	LSTM	LST-	LSSL	TCN	Trans	Re	In	Pyra	Auto	Station			FED	ETS	Flow	DLin-ear	LightTS
Ethanol-Concentration	32.3	43.7	45.2	32.3	32.3	39.9	31.1	28.9	32.7	31.9	31.6	30.8	31.6	32.7	31.2	28.1	33.8	32.6	29.7	35.7	38.4	
FaceDetection	52.9	63.3	64.7	57.7	65.7	66.7	52.8	67.3	68.6	67.0	65.7	68.4	68.0	66.0	66.3	67.6	68.0	67.5	68.6	70.9	70.9	
Handwriting	28.6	15.8	58.8	15.2	25.8	24.6	53.3	32.0	27.4	32.8	29.4	36.7	31.6	28.0	32.5	33.8	27.0	26.1	32.1	37.6	37.6	
Heartbeat	71.7	73.2	75.6	72.2	77.1	72.7	75.6	76.1	77.1	80.5	75.6	74.6	73.7	73.7	71.2	77.6	75.1	75.1	78.0	77.6	77.6	
JapaneseVowels	94.9	86.5	96.2	79.7	98.1	98.4	98.9	98.7	97.8	98.9	98.4	96.2	99.2	98.4	95.9	98.9	96.2	96.2	98.4	97.8	97.8	
PEMS-SF	71.1	98.3	75.1	39.9	86.7	86.1	68.8	82.1	82.7	81.5	83.2	82.7	87.3	80.9	86.0	83.8	75.1	88.4	89.6	88.6	88.6	
SelfRegulationSCP1	77.7	84.6	90.8	68.9	84.0	90.8	84.6	92.2	90.4	90.1	88.1	84.0	89.4	88.7	89.6	92.5	87.3	89.8	91.8	92.5	92.5	
SelfRegulationSCP2	53.9	48.9	53.3	46.6	52.8	52.2	55.6	53.9	56.7	53.3	53.3	50.6	57.2	54.4	55.0	56.1	50.5	51.1	57.2	56.7	56.7	
SpokenArabicDigits	96.3	69.6	71.2	31.9	100.0	100.0	95.6	98.4	97.0	100.0	99.6	100.0	100.0	100.0	100.0	98.8	81.4	100.0	99.0	99.5	99.5	
UWaveGestureLibrary	90.3	75.9	94.4	41.2	87.8	85.9	88.4	85.6	85.6	85.6	83.4	85.9	87.5	85.3	85.0	86.6	82.1	80.3	85.3	87.2	87.2	
Average Accuracy	67.0	66.0	72.5	48.6	71.8	70.9	70.3	71.9	71.5	72.1	70.8	71.1	72.7	70.7	71.0	73.0	67.5	70.4	73.6	74.7	74.7	

Bold and underlined text indicates the best and second-best scores, respectively

TimesNet (73.6%), task-specific deep learning Flowformer (73.0%), and classical method Rocket (72.5%). The results of *TFAS* are obtained from an average search cost of around 48.96 GPU seconds per architecture, meaning that it uses even less search cost on some datasets. Although TimesNet learns hierarchical representations with temporal 2D-variations has shown to be beneficial for the classification task, it cannot obtain the optimal performance due to inadaptability to different time series of the manual configurations.

4.2.4 Anomaly detection

As anomalies are usually hidden in the large-scale data, making the data labeling really challenging, we focus on *unsupervised* time-series anomaly detection to detect the abnormal time points. We compare models on *five* widely-used anomaly detection benchmarks: SMD (Su et al., 2019), NASA MSL/SMAP (Hundman et al., 2018), SWaT (Mathur and Tippenhauer, 2016), and PSM (Abdulaal et al., 2021), covering service monitoring, spacecraft, and water treatment applications. Following Xu et al. (Xu et al., 2022), we split the dataset into consecutive non-overlapping segments by a sliding window and use the classical reconstruction error as the shared anomaly criterion for *all* experiments.

Results. On average, Table 4 shows that *TFAS* also achieves state-of-the-art performance in anomaly detection, outperforming TimesNet and advanced Transformer-based models such as FEDformer and Autoformer. Similarly to the classification task, TimesNet achieves good performance by considering the periodicity, which can implicitly highlight the temporal variations that violate the periodicity, further benefiting anomaly detection. However, it is still inferior to the proposed *TFAS* due to the sub-optimal architecture design by the manual configurations.

4.2.5 Imputation of missing values

The imputation task is widely-used in practical applications because the collected time series can be partially missing due to sensor errors or malfunctions, which makes the downstream analysis more difficult. In this experiment, we use the datasets from the electricity (ETT and Electricity) and weather (Weather) scenarios as our benchmarks where the missing values usually occur. Following Wu et al. (2023), we randomly mask the time points in the ratio of {12.5%, 25%, 37.5%, 50%} to compare the model performance under the different levels of missing values.

Results. Due to the missing time points, the imputation task requires the model to discover underlying temporal patterns from the irregular and partially observed time series. Thus, different missing ratios require different modeling capabilities of deep neural networks. Table 8 demonstrates that *TFAS* achieves consistent state-of-the-art performance in this difficult task, verifying the effectiveness in capturing temporal variation from extremely complicated time series under different missing rates by searching for different architecture configurations. The search only takes 2 GPU hours to obtain the results for each dataset, consisting of four levels of masking ratios (i.e., only 30 GPU minutes per subtask).

Table 8 Full results for the **imputation** task. We randomly mask {12.5%, 25%, 37.5%, 50%} time points to compare the model performance under different missing degrees

Models	Metrics	LSSL		TCN		LSTM		Reformer		LogTrans		Informr		Pyraformer		Autoformer		Stationary	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	12.5%	0.101	0.231	0.510	0.493	0.974	0.780	0.032	0.126	0.041	0.141	0.047	0.155	0.670	0.541	0.034	0.124	0.026	0.107
	25.0%	0.106	0.235	0.518	0.500	1.032	0.807	0.042	0.146	0.044	0.144	0.063	0.180	0.689	0.553	0.046	0.144	0.032	0.119
	37.5%	0.116	0.246	0.516	0.499	0.999	0.792	0.063	0.182	0.052	0.158	0.079	0.200	0.737	0.581	0.057	0.161	0.039	0.131
	50.0%	0.129	0.260	0.519	0.496	0.952	0.763	0.082	0.208	0.063	0.173	0.093	0.218	0.770	0.605	0.067	0.174	0.047	0.145
	Avg	0.113	0.243	0.516	0.497	0.989	0.786	0.055	0.166	0.050	0.154	0.071	0.188	0.717	0.570	0.051	0.151	0.036	0.126
ETTm2	12.5%	0.150	0.298	0.307	0.441	1.013	0.805	0.108	0.228	0.103	0.229	0.133	0.270	0.394	0.470	0.023	0.092	0.021	0.088
	25.0%	0.159	0.306	0.263	0.402	1.039	0.814	0.136	0.262	0.120	0.248	0.135	0.272	0.421	0.482	0.026	0.101	0.024	0.096
	37.5%	0.180	0.321	0.250	0.396	0.917	0.744	0.175	0.300	0.138	0.260	0.155	0.293	0.478	0.521	0.030	0.108	0.027	0.103
	50.0%	0.210	0.353	0.246	0.389	1.140	0.835	0.211	0.329	0.117	0.247	0.200	0.333	0.568	0.560	0.035	0.119	0.030	0.108
	Avg	0.175	0.320	0.267	0.407	1.027	0.800	0.158	0.280	0.120	0.246	0.156	0.292	0.465	0.508	0.029	0.105	0.026	0.099
ETTTh1	12.5%	0.422	0.461	0.599	0.554	1.265	0.896	0.074	0.194	0.229	0.330	0.114	0.234	0.857	0.609	0.074	0.182	0.060	0.165
	25.0%	0.412	0.456	0.610	0.567	1.262	0.883	0.102	0.227	0.207	0.323	0.140	0.262	0.829	0.672	0.090	0.203	0.080	0.189
	37.5%	0.421	0.461	0.628	0.577	1.200	0.867	0.135	0.261	0.210	0.328	0.174	0.293	0.830	0.675	0.109	0.222	0.102	0.212
	50.0%	0.443	0.473	0.648	0.587	1.174	0.849	0.179	0.298	0.230	0.348	0.215	0.325	0.854	0.691	0.137	0.248	0.133	0.240
	Avg	0.425	0.463	0.621	0.571	1.225	0.874	0.123	0.245	0.219	0.332	0.161	0.279	0.843	0.662	0.103	0.214	0.094	0.202
ETTTh2	12.5%	0.521	0.555	0.410	0.494	2.060	1.120	0.163	0.289	0.173	0.308	0.305	0.431	0.976	0.754	0.044	0.138	0.042	0.133
	25.0%	0.487	0.535	0.419	0.490	2.007	1.105	0.206	0.331	0.175	0.310	0.322	0.444	1.037	0.774	0.050	0.149	0.049	0.147
	37.5%	0.487	0.529	0.429	0.498	2.033	1.111	0.252	0.370	0.185	0.315	0.353	0.462	1.107	0.800	0.060	0.163	0.056	0.158
	50.0%	0.484	0.523	0.467	0.529	2.054	1.119	0.316	0.419	0.212	0.339	0.369	0.472	1.193	0.838	0.068	0.173	0.065	0.170
	Avg	0.495	0.536	0.431	0.503	2.039	1.114	0.234	0.352	0.186	0.318	0.337	0.452	1.078	0.792	0.056	0.156	0.053	0.152
Electricity	12.5%	0.217	0.341	0.621	0.620	0.277	0.366	0.190	0.308	0.164	0.296	0.218	0.326	0.297	0.383	0.089	0.210	0.093	0.210
	25.0%	0.219	0.341	0.559	0.585	0.281	0.369	0.197	0.312	0.169	0.299	0.219	0.326	0.294	0.380	0.096	0.220	0.097	0.214
	37.5%	0.223	0.343	0.567	0.588	0.275	0.364	0.203	0.315	0.178	0.305	0.222	0.328	0.296	0.381	0.104	0.229	0.102	0.220
	50.0%	0.229	0.347	0.581	0.597	0.273	0.361	0.210	0.319	0.187	0.312	0.228	0.331	0.299	0.383	0.113	0.239	0.108	0.228
	Avg	0.222	0.343	0.582	0.598	0.277	0.365	0.200	0.314	0.175	0.303	0.222	0.328	0.297	0.382	0.101	0.225	0.100	0.218

Table 8 (continued)

Models	LSSL		TCN		LSTM		Reformer		LogTrans		Informer		Pyraformer		Autoformer		Stationary		
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	
Weather	12.5%	0.036	0.095	0.176	0.287	0.296	0.379	0.031	0.076	0.037	0.072	0.037	0.093	0.140	0.220	0.026	0.047	0.027	0.051
	25.0%	0.042	0.104	0.187	0.293	0.327	0.409	0.035	0.082	0.038	0.074	0.042	0.100	0.147	0.229	0.030	0.054	0.029	0.056
	37.5%	0.047	0.112	0.172	0.281	0.406	0.463	0.040	0.091	0.039	0.078	0.049	0.111	0.156	0.240	0.032	0.060	0.033	0.062
	50.0%	0.054	0.123	0.195	0.303	0.431	0.483	0.046	0.099	0.042	0.082	0.053	0.114	0.164	0.249	0.037	0.067	0.037	0.068
1st Count	Avg	0.045	0.109	0.183	0.291	0.365	0.434	0.038	0.087	0.039	0.077	0.045	0.105	0.152	0.235	0.031	0.057	0.032	0.059
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Models	FEDformer		DLinear		LightTS		ETSformer		TimesNet		TFAS								
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE							
ETTm1	12.5%	0.035	0.135	0.058	0.162	0.075	0.180	0.067	0.188	0.019	0.092	0.017	0.087						
	25.0%	0.052	0.166	0.080	0.193	0.093	0.206	0.096	0.229	0.023	0.101	0.021	0.095						
	37.5%	0.069	0.191	0.103	0.219	0.113	0.231	0.133	0.271	0.029	0.111	0.025	0.104						
	50.0%	0.089	0.218	0.132	0.248	0.134	0.255	0.186	0.323	0.036	0.124	0.033	0.119						
ETTm2	Avg	0.061	0.178	0.093	0.206	0.104	0.218	0.121	0.253	0.027	0.107	0.024	0.101						
	12.5%	0.056	0.159	0.062	0.166	0.034	0.127	0.108	0.239	0.018	0.080	0.017	0.075						
	25.0%	0.080	0.195	0.085	0.196	0.042	0.143	0.164	0.294	0.020	0.085	0.019	0.080						
	37.5%	0.110	0.231	0.106	0.222	0.051	0.159	0.237	0.356	0.023	0.091	0.021	0.086						
ETTth1	50.0%	0.156	0.276	0.131	0.247	0.059	0.174	0.323	0.421	0.026	0.098	0.023	0.092						
	Avg	0.101	0.215	0.096	0.208	0.047	0.151	0.208	0.328	0.022	0.089	0.020	0.083						
	12.5%	0.070	0.190	0.151	0.267	0.240	0.345	0.126	0.263	0.057	0.159	0.044	0.146						
	25.0%	0.106	0.236	0.180	0.292	0.265	0.364	0.169	0.304	0.069	0.178	0.061	0.169						
	37.5%	0.124	0.258	0.215	0.318	0.296	0.382	0.220	0.347	0.084	0.196	0.078	0.188						
	50.0%	0.165	0.299	0.257	0.347	0.334	0.404	0.293	0.402	0.102	0.215	0.094	0.207						
	Avg	0.116	0.246	0.201	0.306	0.284	0.374	0.202	0.329	0.078	0.187	0.069	0.178						

Table 8 (continued)

Models	Metrics	FEDformer				DLinear		LightTS		ETSformer		TimesNet		TFAS	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Electricity	ETTh2	12.5%	0.095	0.212	0.216	0.100	0.216	0.101	0.231	0.187	0.319	<u>0.040</u>	<u>0.130</u>	0.037	0.124
		25.0%	0.137	0.258	0.247	0.127	0.247	0.115	0.246	0.279	0.390	<u>0.046</u>	<u>0.141</u>	0.043	0.136
		37.5%	0.187	0.304	0.276	0.158	0.276	0.126	0.257	0.400	0.465	<u>0.052</u>	<u>0.151</u>	0.049	0.144
		50.0%	0.232	0.341	0.299	0.183	0.299	0.136	0.268	0.602	0.572	<u>0.060</u>	<u>0.162</u>	0.057	0.157
	Avg		0.163	0.279	0.260	0.142	0.260	0.120	0.251	0.367	0.437	<u>0.050</u>	<u>0.146</u>	0.047	0.140
Weather	12.5%	0.107	0.237	0.214	0.214	0.092	0.214	0.102	0.229	0.196	0.321	<u>0.085</u>	<u>0.202</u>	0.066	0.179
	25.0%	0.120	0.251	0.118	0.247	0.118	0.247	0.121	0.252	0.207	0.332	<u>0.089</u>	<u>0.206</u>	0.079	0.195
	37.5%	0.136	0.266	0.144	0.276	0.144	0.276	0.141	0.273	0.219	0.344	<u>0.094</u>	<u>0.213</u>	0.086	0.204
	50.0%	0.158	0.284	0.175	0.305	0.175	0.305	0.160	0.293	0.235	0.357	<u>0.100</u>	<u>0.221</u>	0.095	0.215
	Avg		0.130	0.260	0.261	0.132	0.261	0.131	0.262	0.214	0.339	<u>0.092</u>	<u>0.211</u>	0.082	0.198
1st Count	12.5%	0.041	0.107	0.084	0.084	0.039	0.084	0.047	0.101	0.057	0.141	0.025	0.045	0.025	<u>0.046</u>
	25.0%	0.064	0.163	0.048	0.103	0.048	0.103	0.052	0.111	0.065	0.155	<u>0.029</u>	0.052	0.028	<u>0.053</u>
	37.5%	0.107	0.229	0.057	0.117	0.057	0.117	0.058	0.121	0.081	0.180	0.031	0.057	0.031	0.057
	50.0%	0.183	0.312	0.066	0.134	0.066	0.134	0.065	0.133	0.102	0.207	0.034	0.062	<u>0.037</u>	0.069
	Avg		0.099	0.203	0.110	0.053	0.110	0.056	0.117	0.076	0.171	0.030	0.054	0.030	<u>0.056</u>
1st Count		0	0	0	0	0	0	0	0	0	0	7	7	44	44

Bold and underlined text indicates the best and second-best scores, respectively

Table 9 Experimental results on **ranking performance** between baselines and *TFAS*

Zero-Cost Proxy	Forecasting		Classification		Anomaly Detection		Imputation		Average	
	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ
#Params	0.08	0.06	0.41	0.31	0.04	0.04	0.54	0.41	0.266	0.204
FLOPs	0.13	0.07	0.40	0.30	0.04	0.04	0.52	0.39	0.273	0.200
SynFlow	0.07	0.04	0.31	0.22	0.18	0.09	0.55	0.41	0.277	0.191
Zen-Score	0.21	0.17	0.11	0.06	0.32	0.24	0.21	0.13	0.212	0.149
NTK	0.02	0.02	0.17	0.10	0.08	0.05	0.32	0.21	0.147	0.094
ZiCo	0.23	0.16	0.48	0.35	0.33	0.23	0.55	0.41	0.396	0.288
<i>TFAS</i>	0.23	0.17	0.49	0.36	0.35	0.27	0.57	0.42	0.411	0.306
Improvement (%)	+0.06	+8.10	+1.77	+2.48	+7.28	+17.89	+4.91	+2.39	+3.75	+6.28

Bold and underlined text indicates the best and second-best scores, respectively
Note that lower (a) forecasting and (d) imputation errors (i.e., MSE) are ranked higher

4.3 Ranking correlation analysis

As it is essential to analyze the ranking consistency when evaluating the zero-shot proxy for NAS. We thus build a small benchmark by randomly sampling architectures from the search space, ranging from 54 to 672 samples, given the different computational costs in full training across different tasks. Here, for comprehensive comparisons, we select three representative *data-independent* (#Params, SynFlow (Tanaka et al., 2020), and Zen-Score (Lin et al., 2021)) and three representative *data-dependent* (FLOPs, NTK (Shu et al., 2022), and ZiCo (Li et al., 2023)) baselines. As in the recent studies (Krishnakumar et al., 2022; Li et al., 2023), we calculate the Kendall's τ and Spearman's ρ rank correlations between the zero-shot proxy scores and the ground truth performance. Table 9 presents the results of this experiment, indicating consistent improvements by up to **17%** over the second-best baseline. Hence, *TFAS* are indeed helpful for the zero-shot proxy to better score the untrained architectures across various datasets and tasks. Interestingly, we notice the relatively weak correlation between the architecture ranking and the actual performance on the forecasting tasks, which could be a reason *TFAS* cannot significantly achieve state-of-the-art performance in short-term forecasting and few cases in long-term forecasting tasks.

4.4 Ablation study

This ablation study examines the effectiveness of the proposed *time-frequency aware scoring* on architecture ranking performance. Table 10 presents impact of time-frequency aware scoring on ranking accuracy across four tasks: forecasting, classification, anomaly detection, and imputation. The inclusion of time-domain and frequency-domain features was evaluated independently and in combination. For forecasting and classification, incorporating both domains improved the ranking accuracy as evidenced by higher Spearman's ρ and Kendall's τ values (e.g., for forecasting, $\rho = 0.23$ and $\tau = 0.17$). Similarly, in anomaly detection and imputation tasks, the combined approach consistently outperformed single-domain configurations, achieving the highest scores (e.g., for imputation, $\rho = 0.57$ and $\tau = 0.42$). This demonstrates the complementary nature of time-domain and frequency-domain features, reinforcing the importance of integrating both to enhance ranking accuracy across diverse tasks.

5 Limitations and future work

To quantify the contribution of our time-frequency aware scoring, we currently test our *TFAS* only on the CNN-based architecture to align with the capabilities of the base proxy. However, our method may require further modifications when applied to other types of neural networks (e.g., Transformers) and zero-shot proxies.

As future work, we expect to explore zero-shot proxies that work effectively across different types of neural networks for time-series analysis, potentially focusing on improving the ranking quality of forecasting tasks. For example, this could involve using alternative

Table 10 Effect of time-frequency aware scoring on ranking accuracy

Scoring Variations		Forecasting		Classification		Anomaly Detection		Imputation	
Time Domain	Frequency Domain	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ	Spearman's ρ	Kendall's τ
-	-	0.23	0.16	0.48	0.35	0.33	0.23	0.55	0.41
✓	-	0.21	0.16	0.49	0.35	0.32	0.24	0.56	0.41
-	✓	0.23	0.16	0.45	0.33	0.35	0.27	0.55	0.41
✓	✓	0.23	0.17	0.49	0.36	0.35	0.27	0.57	0.42

time-series features or more sophisticated task embeddings. Overall, we anticipate that our work will contribute to further reducing the energy consumption and CO₂ emissions associated with expensive model training and manual trial-and-error processes on numerous datasets and analysis tasks.

6 Conclusions

This paper presents *TFAS*, a novel zero-shot neural architecture search framework for general time-series analysis. Motivated by the need of consistent zero-shot proxy across different tasks, *TFAS* enhances the existing zero-shot proxy with time-frequency aware scoring, making it become aware of the underlying characteristics of input time-series data when scoring neural architectures. Extensive experiments demonstrate that the architectures discovered by *TFAS* perform better than state-of-the-art hand-crafted models with the improvement of about 0.2–23.6%.

Appendix A: Extra experimental results

As *TFAS* clearly outperforms most baselines except TimesNet, we repeat the experiments for *TFAS* and TimesNet *three* times and then report the average with the standard deviation to provide more reliable results as in the following figures for comparison: long-term forecasting in Figs. 4 and 5, short-term forecasting in Fig. 6, classification in Fig. 7, anomaly detection in Fig. 8, and imputation in Figs. 9 and 10.

According to the results, we can notice that models found by *TFAS* outperform TimesNet on most tasks and datasets with state-of-the-art performance, except for short-term forecasting. Here, as shown in Table 10, we conjecture that *TFAS* fails to attain state-of-the-art performance in short-term forecasting and part of long-term forecasting due to the comparatively weak correlation between architecture ranking and actual performance in the forecasting tasks.

In future work, we will explore how to improve the ranking quality of the forecasting tasks. For example, exploring other types of time-series features or using more sophisticated task embedding relevant to forecasting challenges.

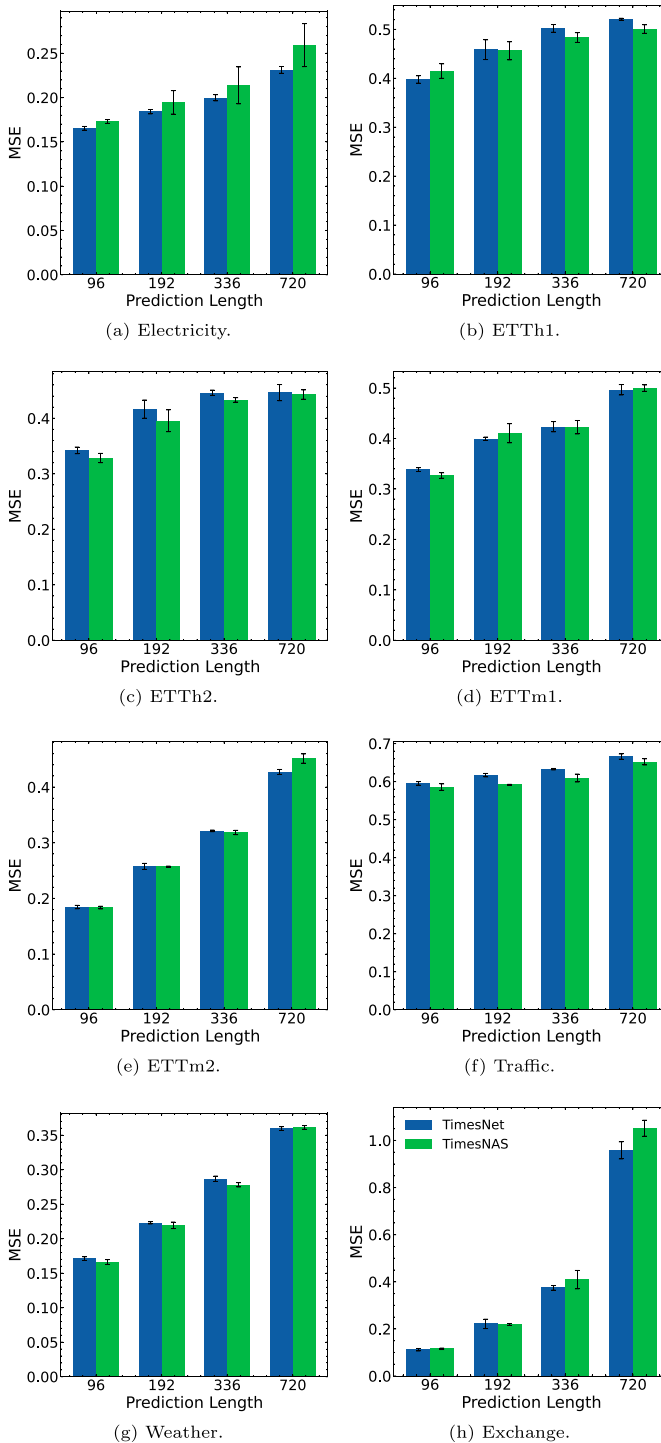


Fig. 4 Performance comparison for the long-term forecasting task in the MSE metric between *TFAS* and TimesNet averaged from three different runs

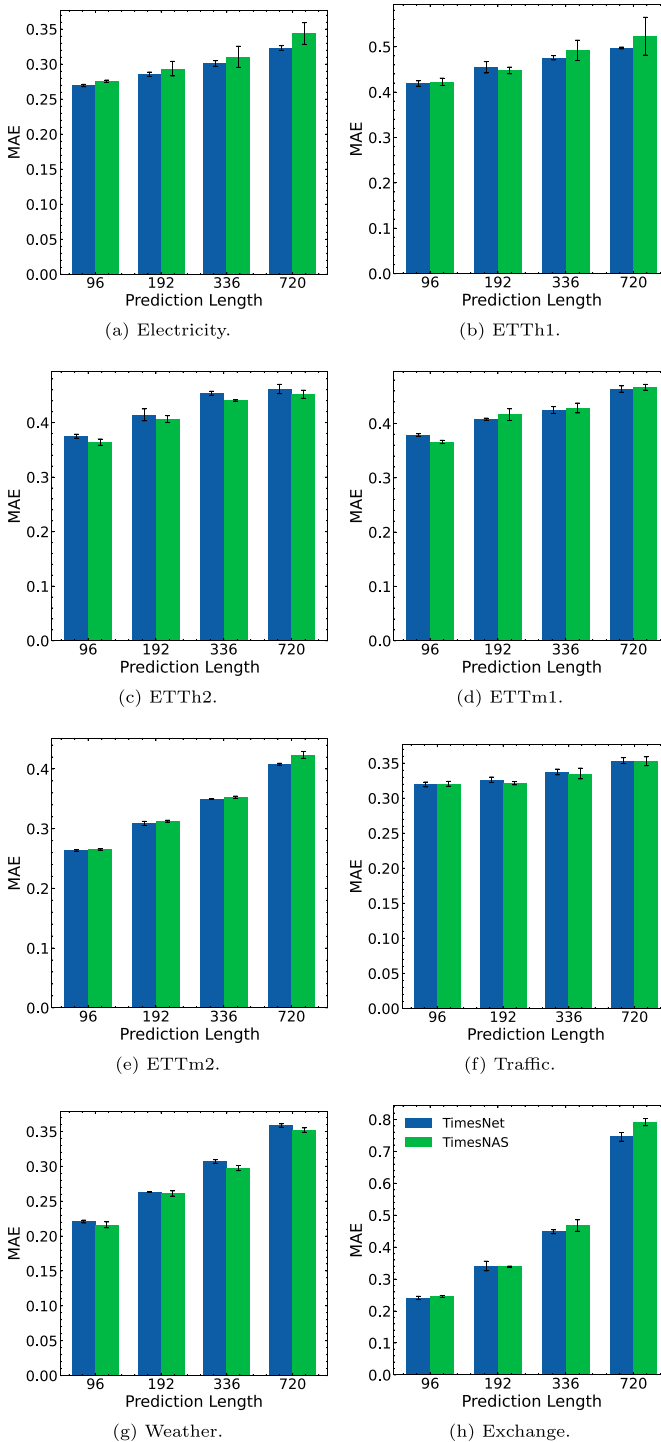


Fig. 5 Performance comparison for the long-term forecasting task in the MAE metric between *TFAS* and TimesNet averaged from three different runs

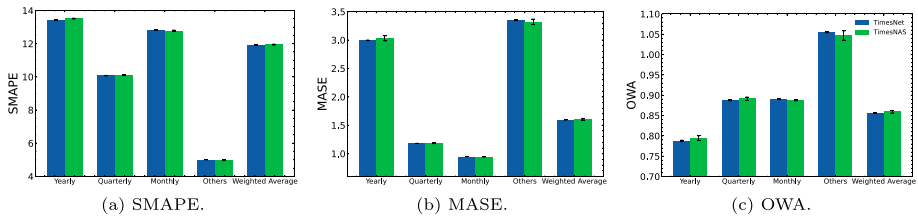


Fig. 6 Performance comparison for the short-term forecasting task on the M4 benchmark between *TFAS* and TimesNet averaged from three different runs

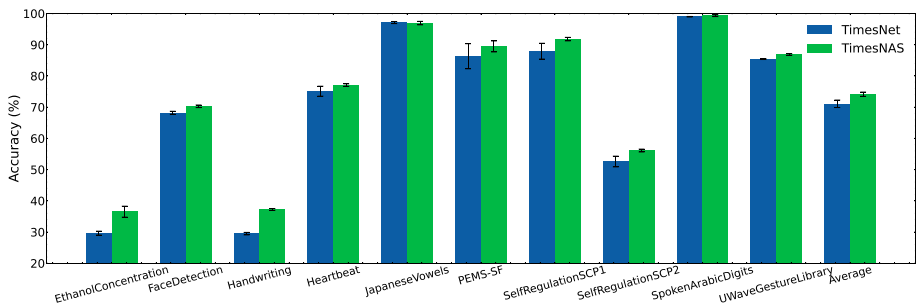


Fig. 7 "Performance comparison for the classification task on 10 UEA datasets between *TFAS* and TimesNet in the accuracy metric (%) averaged from three different runs

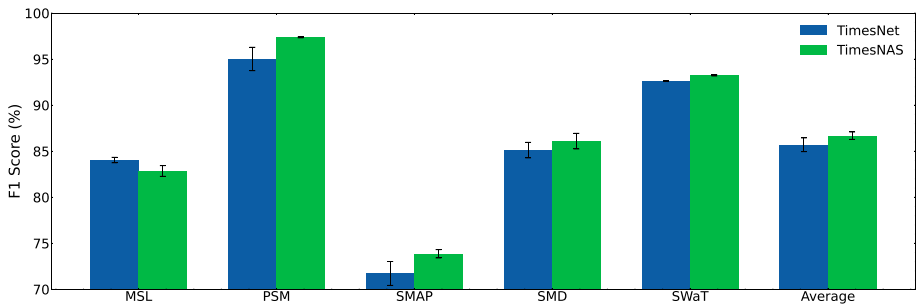


Fig. 8 Performance comparison for the anomaly detection task between *TFAS* and TimesNet in the point-adjust F1-score metric (%) averaged from three different runs

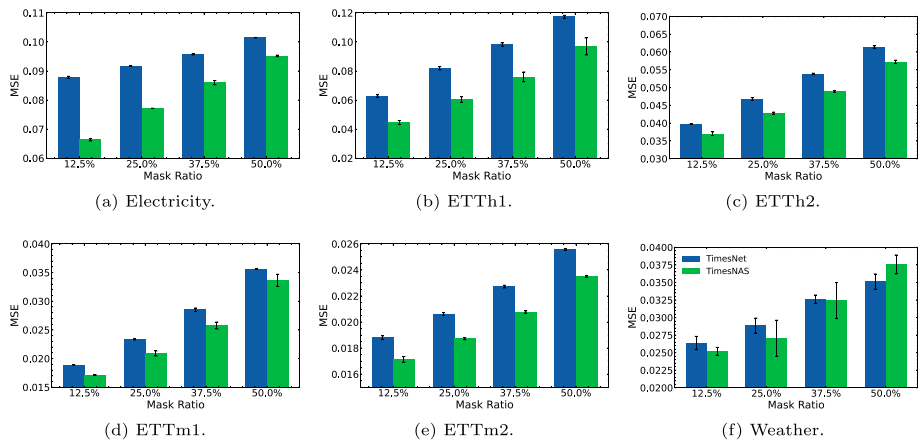


Fig. 9 Performance comparison for the imputation task in the MSE metric between *TFAS* and *TimesNet* averaged from three different runs

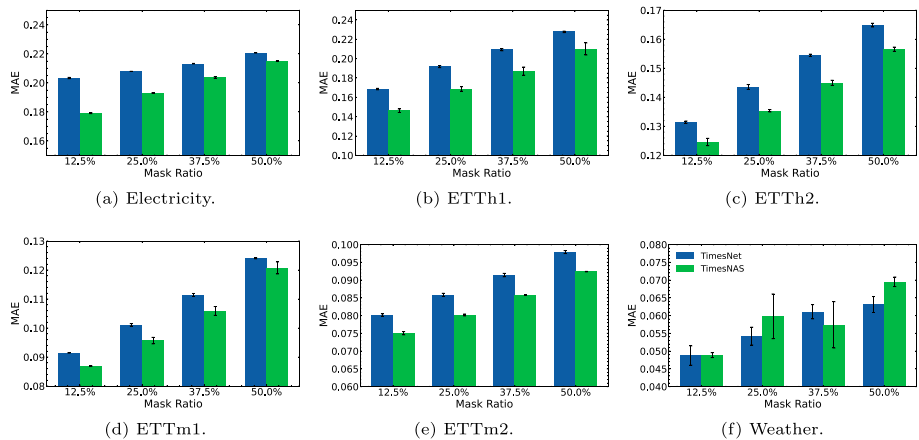


Fig. 10 Performance comparison for the imputation task in the MAE metric between *TFAS* and *TimesNet* averaged from three different runs

Acknowledgements This work was supported by Institute of Information & Communications Technology Planning & Evaluation, (IITP) grant funded by the Korea government, (MSIT) (No. RS-2020-II200862, DB4DL: High-Usability and Performance In-Memory Distributed DBMS for Deep Learning, 50% and No. RS-2022-II220157, Robust, Fair, Extensible Data-Centric Continual Learning, 50%).

Author contributions P.T. conceptualized the study, developed the methodology, conducted the investigation, implemented the software, performed validation and visualization, and wrote the original draft of the manuscript. J.G.L. supervised the project, secured funding, provided resources, and contributed to the review and editing of the manuscript. All authors reviewed the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Abdelfattah, M. S., Mehrotra, A., Dudziak, Ł., & Lane, N. D. (2021). Zero-cost proxies for lightweight NAS. *Iclr* (pp. 1–17).
- Abdulaal, A., Liu, Z., & Lancewicki, T. (2021). Practical approach to asynchronous multivariate time series anomaly detection and localization. *Kdd* (pp. 2485–2494).
- Anderson, O. D., & Kendall, M. G. (1976). Time-series. 2nd edn. *The Statistician*, 25, 308.
- Bagnall, A., Dau, H. A., Lines, J., Flynn, M., Large, J., Bostrom, A., Keogh, E. (2018). The UEA multivariate time series classification archive, 2018. [arXiv:1811.00075](https://arxiv.org/abs/1811.00075).
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR*, 13(2).
- Challu, C., Olivares, K. G., Oreshkin, B. N., Garza, F., Mergenthaler, M., & Dubrawski, A. (2023). N-HiTS: Neural hierarchical interpolation for time series forecasting. *Aaaai* (pp. 6989–6997).
- Choi, K., Yi, J., Park, C., & Yoon, S. (2021). Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines. *IEEE Access*.
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition. *Journal of Official Statistics*, 6(1), 3–73.
- Cuturi, M. (2011). Fast global alignment kernels. *ICML* (pp. 929–936).
- Dempster, A., Petitjean, F., & Webb, G. I. (2020). ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels. *DMKD*, 34(5), 1454–1495.
- Deng, D., Karl, F., Hutter, F., Bischl, B., & Lindauer, M. (2022). Efficient automated deep learning for time series forecasting. *Ecml pkdd* (pp. 664–680).
- Foumani, N. M., Miller, L., Tan, C. W., Webb, G. I., Forestier, G., & Salehi, M. (2023). Deep learning for time series classification and extrinsic regression: A current survey. [arXiv:2302.02515](https://arxiv.org/abs/2302.02515).
- Franceschi, J.-Y., Dieuleveut, A., & Jaggi, M. (2019). Unsupervised scalable representation learning for multivariate time series. *Neurips* (pp. 1–12).
- Galván, E., & Mooney, P. (2021). Neuroevolution in deep neural networks: Current trends and future challenges. *IEEE TAI*, 2(6), 476–493.
- Gu, A., Goel, K., & Re, C. (2022). Efficiently modeling long sequences with structured state spaces. *Iclr* (pp. 1–27).
- Haq, I. U., Lee, B. S., & Rizzo, D. M. (2024). Transnas-tsad: harnessing transformers for multi-objective neural architecture search in time series anomaly detection. *Neural Computing and Applications*, 1–23.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Hundman, K., Constantinou, V., Laporte, C., Colwell, I., & Soderstrom, T. (2018). Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. *Kdd*, pp. 387–395.
- Jiang, T., Wang, H., & Bie, R. (2023). MeCo: zero-shot nas with one data and single forward pass via minimum eigenvalue of correlation. *Neurips*, vol. 36.
- Kitaev, N., Kaiser, L., & Levskaya, A. (2020). Reformer: The efficient transformer. *Iclr*, pp. 1–12.
- Krishnakumar, A., White, C., Zela, A., Tu, R., Safari, M., & Hutter, F. (2022). NAS-bench-suite-zero: Accelerating research on zero cost proxies. *Neurips*, pp. 28037–28051.
- Lai, G., Chang, W.-C., Yang, Y., & Liu, H. (2018). Modeling long-and short-term temporal patterns with deep neural networks. *Acm sigir*, pp. 95–104.
- Lai, Z., Zhang, D., Li, H., Jensen, C.S., Lu, H., & Zhao, Y. (2023). Lightcts: A lightweight framework for correlated time series forecasting. *Proc. ACM Manag. Data*, 1(2).
- Li, G., Hoang, D., Bhardwaj, K., Lin, M., Wang, Z., & Marculescu, R. (2024). Zero-shot neural architecture search: Challenges, solutions, and opportunities. *IEEE TPAMI*.
- Li, G., Yang, Y., Bhardwaj, K., & Marculescu, R. (2023). ZiCo: Zero-shot NAS via inverse coefficient of variation on gradients. *Iclr*, pp. 1–31.
- Li, L., & Talwalkar, A. (2020). Random search and reproducibility for neural architecture search. *Uai*, pp. 367–377.
- Li, T., Zhang, J., Bao, K., Liang, Y., Li, Y., & Zheng, Y. (2020). AutoST: Efficient neural architecture search for spatio-temporal prediction. *Kdd*, pp. 794–802.
- Lim, B., & Zohren, S. (2021). Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194).

- Lin, M., Wang, P., Sun, Z., Chen, H., Sun, X., Qian, Q., Jin, R. (2021). Zen-NAS: A zero-shot nas for high-performance image recognition. *Iccv*, pp. 347–356.
- Lindauer, M., & Hutter, F. (2020). Best practices for scientific research on neural architecture search. *JMLR*, 21(243), 1–18.
- Liu, H., Simonyan, K., & Yang, Y. (2018). DARTS: Differentiable architecture search. *Iclr*.
- Liu, S., Yu, H., Liao, C., Li, J., Lin, W., Liu, A. X., & Dustdar, S. (2022). Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. *Iclr*, pp. 1–20.
- Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., & Long, M. (2024). iTransformer: Inverted transformers are effective for time series forecasting. *Iclr*.
- Liu, Y., Wu, H., Wang, J., & Long, M. (2022). Non-stationary Transformers: Exploring the stationarity in time series forecasting. *Neurips*, pp. 1–13.
- Luo, D., & Wang, X. (2024). ModernTCN: A modern pure convolution structure for general time series analysis. *Iclr*.
- Makridakis, S. (2018). *M4 dataset*. <https://github.com/M4Competition/M4-methods/tree/master/Dataset>.
- Mathur, A. P., & Tippenhauer, N. O. (2016). SWaT: A water treatment testbed for research and training on ics security. *Cyswater*, pp. 31–36.
- Mellor, J., Turner, J., Storkey, A., & Crowley, E. J. (2021). Neural architecture search without training. *ICML*, pp. 7588–7598.
- Miao, X., Wu, Y., Wang, J., Gao, Y., Mao, X., & Yin, J. (2021). Generative semi-supervised learning for multivariate time series imputation. *AAAI*, pp. 8983–8991.
- Mo, T., & Liu, B. (2021). Encoder-decoder neural architecture optimization for keyword spotting. [arXiv:2106.02738](https://arxiv.org/abs/2106.02738).
- Nam, Y., Trirat, P., Kim, T., Lee, Y., & Lee, J.-G. (2023). Context-aware deep time-series decomposition for anomaly detection in businesses. *ECML PKDD*, pp. 330–345.
- Ning, X., Tang, C., Li, W., Zhou, Z., Liang, S., Yang, H., & Wang, Y. (2021). Evaluating efficient performance estimators of neural architectures. *Neurips*, pp. 12265–12277.
- Oreshkin, B. N., Carпов, D., Chapados, N., & Bengio, Y. (2020). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. *Iclr*, pp. 1–31.
- Pan, Z., Ke, S., Yang, X., Liang, Y., Yu, Y., Zhang, J., & Zheng, Y. (2021). AutoSTG: Neural architecture search for predictions of spatio-temporal graph. *WWW*, pp. 1846–1855.
- Rakhshani, H., Fawaz, H. I., Idoumghar, L., Forestier, G., Lepagnot, J., Weber, J., Muller, P.-A. (2020). Neural architecture search for time series classification. *IJCNN*, pp. 1–8.
- Real, E., Aggarwal, A., Huang, Y., & Le, Q. V. (2019). Regularized evolution for image classifier architecture search. *AAAI*, pp. 4780–4789.
- Ren, Y., Li, L., Yang, X., & Zhou, J. (2022). AutoTransformer: Automatic transformer architecture design for time series classification. *PAKDD*, pp. 143–155.
- Shin, Y., Yoon, S., Song, H., Park, D., Kim, B., Lee, J.-G., & Lee, B. S. (2023). Context consistency regularization for label sparsity in time series. *ICML*, pp. 31579–31595.
- Shu, Y., Cai, S., Dai, Z., Ooi, B. C., & Low, B. K. H. (2022). NASI: Label- and data-agnostic neural architecture search at initialization. *Iclr*, pp. 1–25.
- Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., & Pei, D. (2019). Robust anomaly detection for multivariate time series through stochastic recurrent neural network. *KDD*, pp. 2828–2837.
- Tanaka, H., Kunin, D., Yamins, D. L., & Ganguli, S. (2020). Pruning neural networks without any data by iteratively conserving synaptic flow. *Neurips*, pp. 6377–6389.
- Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
- Tenorio, M., & Lee, W.-T. (1988). Self organizing neural networks for the identification problem. *Nips*, pp. 57–64.
- Trirat, P., & Lee, J.-G. (2024). PASTA: Neural architecture search for anomaly detection in multivariate time series. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 1–16.
- Trirat, P., Nam, Y., Kim, T., & Lee, J.-G. (2023). Anoviz: a visual inspection tool of anomalies in multivariate time series. *Proceedings of the AAAI conference on artificial intelligence* (Vol. 37, pp. 16488–16490).
- Trirat, P., Shin, Y., Kang, J., Nam, Y., Na, J., Bae, M., Lee, J.-G. (2024). Universal time-series representation learning: A survey. [arXiv preprint arXiv:2401.03717](https://arxiv.org/abs/2401.03717).
- Trirat, P., Yoon, S., & Lee, J.-G. (2023). MG-TAR: Multi-view graph convolutional networks for traffic accident risk prediction. *IEEE T-ITS*, 24(4), 3779–3794.
- UCI (2015). Electricity. <https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014>.
- Wang, S., Wu, H., Shi, X., Hu, T., Luo, H., Ma, L., Zhou, J. (2024). TimeMixer: Decomposable multiscale mixing for time series forecasting. *Iclr*.
- Wang, Y., Wu, H., Dong, J., Liu, Y., Long, M., & Wang, J. (2024). Deep time series models: A comprehensive survey and benchmark. [arXiv preprint arXiv:2407.13278](https://arxiv.org/abs/2407.13278).

- Wang, Y., Yang, Y., Chen, Y., Bai, J., Zhang, C., Su, G., Zhou, L. (2020). TextNAS: A neural architecture search space tailored for text representation. *Aaai*, pp. 9242–9249.
- Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., & Sun, L. (2023). Transformers in time series: A survey. *Ijcai*, pp. 6778–6786.
- Wetterstation (2020). *Weather*. <https://www.bgc-jena.mpg.de/wetter>.
- White, C., Khodak, M., Tu, R., Shah, S., Bubeck, S., & Dey, D. (2022). A deeper look at zero-cost proxies for lightweight nas. ICLR Blog Track, Retrieved from <https://iclr-blog-trwewlegement.github.io/2022/03/25/zero-cost-proxies>
- White, C., Neiswanger, W., & Savani, Y. (2021). BANANAS: Bayesian optimization with neural architectures for neural architecture search. *Aaai* (pp. 10293–10301).
- White, C., Safari, M., Sukthanker, R., Ru, B., Elsken, T., Zela, A., Hutter, F. (2023). Neural architecture search: Insights from 1000 papers. [arXiv:2301.08727](https://arxiv.org/abs/2301.08727).
- White, C., Zela, A., Ru, R., Liu, Y., & Hutter, F. (2021). How powerful are performance predictors in neural architecture search? *Neurips*, pp. 28454–28469.
- Woo, G., Liu, C., Sahoo, D., Kumar, A., & Hoi, S. (2022). ETSformer: Exponential smoothing transformers for time-series forecasting. [arXiv:2202.01381](https://arxiv.org/abs/2202.01381).
- Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., & Long, M. (2023). TimesNet: Temporal 2d-variation modeling for general time series analysis. *Iclr*, pp. 1–23.
- Wu, H., Wu, J., Xu, J., Wang, J., & Long, M. (2022). Flowformer: Linearizing transformers with conservation flows. *icml*, pp. 24226–24242.
- Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Neurips*, pp. 22419–22430.
- Wu, X., Wu, X., Yang, B., Zhou, L., Guo, C., Qiu, X., Jensen, C. S. (2024). Autocts++: zero-shot joint neural architecture and hyperparameter search for correlated time series forecasting. *The VLDB Journal*, 1–28.
- Wu, X., Zhang, D., Guo, C., He, C., Yang, B., & Jensen, C. S. (2021). Autocts: Automated correlated time series forecasting. *Proceedings of the VLDB Endowment*, 15(4), 971–983.
- Xiao, Z., Xu, X., Xing, H., Qu, R., Song, F., & Zhao, B. (2021). RNTS: Robust neural temporal search for time series classification. *Ijcn*, pp. 1–8.
- Xu, J., Wu, H., Wang, J., & Long, M. (2022). Anomaly transformer: Time series anomaly detection with association discrepancy. *Iclr*.
- Yi, K., Zhang, Q., Cao, L., Wang, S., Long, G., Hu, L., Xiong, H. (2023). A survey on deep learning based time series analysis with frequency transformation. [arXiv:2302.02173](https://arxiv.org/abs/2302.02173).
- Yue, Z., Wang, Y., Duan, J., Yang, T., Huang, C., Tong, Y., & Xu, B. (2022). TS2Vec: Towards universal representation of time series. *Aaai* (pp. 8980–8987).
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Aaai*, pp. 11121–11128.
- Zhang, C., Zhou, T., Wen, Q., & Sun, L. (2022). TFAD: A decomposition time series anomaly detection architecture with time-frequency analysis. *Cikm*, pp. 2497–2507.
- Zhang, T., Zhang, Y., Cao, W., Bian, J., Yi, X., Zheng, S., & Li, J. (2022). Less is more: Fast multivariate time series forecasting with light sampling-oriented MLP structures. [arXiv:2207.01186](https://arxiv.org/abs/2207.01186).
- Zhang, X., Zhao, Z., Tsiligkaridis, T., & Zitnik, M. (2022). Self-supervised contrastive pre-training for time series via time-frequency consistency. *Neurips*, pp. 3988–4003.
- Zhang, Y., & Yan, J. (2023). Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. *Iclr*, pp. 1–21.
- Zhang, Z., & Jia, Z. (2022). GradSign: Model performance inference with theoretical insights. *Iclr*.
- Zhang, Z., Wang, X., Guan, C., Zhang, Z., Li, H., & Zhu, W. (2023). AutoGT: Automated graph transformer architecture search. *Iclr*, pp. 1–14.
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *Aaai*, pp. 11106–11115.
- Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., & Jin, R. (2022). FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. *icml*, pp. 27268–27286.
- Zoph, B., & Le, Q. V. (2017). Neural architecture search with reinforcement learning. *Iclr*.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.